PR #215: docs(getting-started): add a Serve Models guide built on the tt CLI and TT-Studio

https://github.com/tenstorrent/tenstorrent.github.io/pull/215
Branch `deploy-models-tt-cli` → `getting-started-tt-cli` · author ndalyTT · head `00df4bef`

Rendered review copy of the pages this PR changes, built with Sphinx from the PR head (commit `00df4be`; a stacked PR already contains its base branch), generated 2026-11-04. Pages render in the site theme with the navigation removed; code blocks wrap instead of scrolling.

Commits (13)

`4038f872`	2026-11-04	docs(getting-started): replace Deploy LLMs with Deploy Models built o…
`1bd74977`	2026-11-04	docs(getting-started): rename Deploy Models to Serve Models and keep …
`25ad14e5`	2026-11-04	docs(getting-started): point Serve Models away from the removed insta…
`d713d366`	2026-11-04	docs(getting-started): remove the Deploy LLMs page from Get Started
`459fac90`	2026-11-04	docs(getting-started): name the three serving paths behind tt serve
`9291ba11`	2026-11-04	docs(getting-started): drop the row-by-row explanation under Check wh…
`c59fbac4`	2026-11-04	docs(getting-started): drop the serve-timing paragraph from Serve Models
`b4d1a978`	2026-11-04	docs(getting-started): unnumber the sub-steps on the Serve Models page
`97b34fac`	2026-11-04	docs(getting-started): drop the clip-timing sentence from Serve Models
`b7f64aeb`	2026-11-04	docs(getting-started): shorten the TT-Studio step on the Serve Models…
`19b8493b`	2026-11-04	docs(getting-started): shorten the CLI step on the Serve Models page
`2fd57e0e`	2026-11-04	docs(getting-started): Wormhole notes and tested behaviour on the Ser…

00df4bef	2026-11-04	docs(getting-started): drop the EXPERIMENTAL sentence from the Wormho…

Files changed (10)

core/_extra/AGENTS.md	+1	-1
core/_extra/llms.txt	+2	-2
core/_templates/home.html	+3	-3
core/conf.py	+2	-0
core/getting-started/README.md	+3	-2
core/getting-started/serve-models.md	+362	-0
core/getting-started/tt-software-stack.md	+1	-1
core/getting-started/vLLM-servers.md	+0	-214
core/index.rst	+1	-1
shared/_templates/layout.html	+1	-1

Pages in this PDF

1. getting-started/serve-models.html
2. index.html
3. getting-started/README.html
4. getting-started/tt-software-stack.html

Serve Models

This page shows how to serve a model on your Tenstorrent system and start sending it requests. You have two surfaces to work from, and both draw on the same catalog of models that Tenstorrent validates on each hardware generation:

- **The `tt` CLI.** `tt serve <model>` gives you an OpenAI-compatible API endpoint in one command. This is the path for applications, scripts, agents, and anything else that talks to an API.
- **TT-Studio.** A web application that serves the same models and gives you a browser interface for them: chat, image and video generation, speech-to-text, object detection, and more. This is the path when you want to try a model from a browser, or hand it to people who do not use a terminal.

The flow is the same whatever kind of model you serve. The catalog is not limited to large language models: it includes diffusion models for image and video generation, speech and audio models, decision models, and tabular foundation models, and `tt serve` and TT-Studio treat each of them the same way. Under the hood, `tt serve` runs TT-Inference-Server for the models in Tenstorrent's tested catalog, hands the few models only TT-Studio carries to TT-Studio, and runs community bundles published on the Hugging Face Hub through tt-model-manager. You do not need to clone or configure any of them yourself.

The Same Steps for Every Model

Serving any model in the catalog takes the same five steps, and each is one command:

Step	Command	What it does
1. See what runs on your system	`tt model list`	Lists the models validated for th
2. Check what the model needs	`tt model info <model>`	Reports the disk and memory th
3. Serve it	`tt serve <model>`, or `tt serve <model> --studio` for TT-Studio	Downloads the weights and the
4. Send a request	`curl http://localhost:20000/v1/<route>`	Every server speaks the OpenAI
5. Manage and stop	`tt model ps`, `tt model logs <model>`, `tt model stop <model>`	Shows what is running, streams

The commands do not change from one model to the next. What changes is what they report: the resources a model needs, the stages `tt serve` passes through while it loads, the route you call, and what comes back. The rest of this page walks the steps with `Wan2.2-T2V-A14B-Diffusers`, the Wan 2.2 text-to-video diffusion model, as the example, and says where another kind of model behaves differently. The example needs four chips, so it runs on a TT-QuietBox 2, a TT-LoudBox, or a Galaxy; on a single-chip board, follow the same steps with a model from your own list.

Before You Begin

Important

- This guide assumes you have completed Installing Tenstorrent Software: `tt update` has run, `tt device status` lists your devices, and Docker runs without `sudo`.
- You are logged in to Hugging Face with `tt model login`. Most models download their weights from the Hub, and the login is needed even for models that have no license gate. `tt model login` writes the same token store as the Hub's own `hf auth login`, which is the command `tt serve` names when the token is missing; either one satisfies it, as does an exported `HF_TOKEN`.
- Disk space depends on the model. `tt model info <model>` reports the minimum for each one, from 15 GB for a speech or image model to 160 GB for a 70B-parameter LLM. Downloaded weights are cached under `~/.cache/huggingface` and shared by every path that serves the model.

(Wormhole™ only)

Wormhole multi-card systems need a mesh topology

On a Wormhole TT-QuietBox® or TT-LoudBox™, run `tt-topology -ls` and confirm it reports `Mesh`. If it reports `Isolated`, complete Step 4 of Installing Tenstorrent Software first; it takes about two minutes and resets the chips. Without it, every serve on this page stops at `validating system software` with `ModelSpec requires SystemTopology.MESH, detected ISOLATED`.

Step 1: Choose a Model

See what runs on your system

`tt model list` shows every model that is validated for the hardware it detects, from two sources: the released catalog that Tenstorrent ships and tests, and community bundles that anyone has published for Tenstorrent hardware.

```
tt model list
```

Each row names the model, its `source` (`tt-inference-server` or `tt-studio` for the catalog, `HuggingFace` for a community bundle), the `engine` that runs it (`vLLM` for language models, `media` for image, video, audio, and embedding models, and `forge` for the vision and embedding models that run through TT-Forge), the smallest board or mesh each capability needs, and whether the weights are already on disk. Narrow the list to one kind of model with `--type`:

Kind of model	`--type`	Examples in the catalog
Large language models	`llm`	`Qwen3.8-27B`, `Llama-3.3-70B-Instruct`, `gpt-oss-120b`
Vision-language models	`vlm`	`Qwen2.5-VL-7B-Instruct`, `Llama-3.2-11B-Vision-Instruct`
Image generation (diffusion)	`image`	`FLUX.1-schnell`, `FLUX.1-dev`, `stable-diffusion-xl-base-1.0`
Video generation (diffusion)	`video`	`Wan2.2-T2V-A14B-Diffusers`, `mochi-1-preview`
Speech recognition	`audio`	`whisper-large-v3`, `distil-large-v3`
Text to speech	`text_to_speech`	`speecht5_tts`
Vision classification and detection	`cnn`	`yolox_nano`, `resnet-50`, `segformer`
Embeddings	`embedding`	`bge-m3`, `Qwen3-Embedding-4B`

Wormhole

On a Wormhole system, a model can be listed and still be refused by `tt serve`. Each model is pinned to a container image, and the inference server only runs images at or above a minimum version; when the pin is older, the serve stops at `host ready` with `Image v0.4.0 is not supported (need v0.11.0+)`. To check a model before you download anything, run `tt serve <model> --dry-run` and compare the version at the start of its `docker image` tag with the minimum the server requires. The same pin is listed on each model's page in the tt-inference-server model support tables, and the minimum is set in the server's setup validation.

The example model on this page is a video model. Listing that kind on a TT-QuietBox 2 gives a list of one:

```
tt model list --type video
```

```
Models (tt catalog + community) for p300x2 (detected — `tt model list --all` for every
                                device/bundle)
┌──────────────────────────┬──────────────────────┬──────────┬──────────────────┬─────────┐
│ name                     │ source               │ engine   │ serving profiles │ weights │
├──────────────────────────┼──────────────────────┼──────────┼──────────────────┼─────────┤
│ Wan2.2-T2V-A14B-Diffusers │ tt-inference-server │ media    │ p150x4, p300x2   │ —       │
└──────────────────────────┴──────────────────────┴──────────┴──────────────────┴─────────┘
```

A model that is known not to work on your board is left out. `tt model list --all` shows every model on every device, and `--hw p150x4` shows the list for another configuration without touching the hardware. `--catalog` and `--community` show one source at a time.

Check what a model needs

Before you commit to a download, inspect the model. `tt model info <model>` reports where the weights come from, which surfaces can serve the model, what it needs from the host, and how it is validated on each hardware configuration. For the example model:

```
tt model info Wan2.2-T2V-A14B-Diffusers
```

```
                                    Wan2.2-T2V-A14B-Diffusers
┌──────────────────────┬──────────────────────────────────────────────────────────────────────────────────────────────
──────────┐
│ hf_repo              │ Wan-AI/Wan2.2-T2V-A14B-Diffusers
│
│ type                 │ video
│
│ engines              │ media
│
│ via                  │ inference-server, studio
│
│ servable             │ yes — `tt serve Wan2.2-T2V-A14B-Diffusers` (`--studio` deploys it with
TT-Studio) │
│ parameters           │ 14B
│
│ min disk             │ 60 GB
│
│ min ram              │ 32 GB
│
│ cached               │ no
│
│ on t3k               │ media — COMPLETE, max context 65536
│
│ on galaxy            │ media — COMPLETE, max context 65536
│
│ on p150x4            │ media — COMPLETE, max context 65536
│
│ on p150x8            │ media — COMPLETE, max context 65536
│
│ on p300x2            │ media — COMPLETE, max context 65536
│
│ on blackhole_galaxy  │ media — COMPLETE, max context 65536
│
└──────────────────────┴──────────────────────────────────────────────────────────────────────────────────────────────
──────────┘
```

Get access to gated models

Some model owners gate their weights behind a license. Meta's Llama models, Google's Gemma models, and Black Forest Labs' FLUX.1 models all require you to accept their terms on the model's Hub page before you can download them, and the owner may take days to grant access. Qwen, Wan, and Whisper models are not gated.

To get access, open the `hf_repo` page that `tt model info` shows, accept the license, and make sure the account you accepted with is the one you used for `tt model login`. Until access is granted, serving a gated model stops at the download:

```
  ✓ host ready  tt-inference-server 0.25.0
  ✗ downloading weights  black-forest-labs/FLUX.1-schnell
╭──────────────────────────── error [TOOL_FAILED] ─────────────────────────────╮
│ FLUX.1-schnell stopped before the server was ready.                          │
│ AssertionError: ⛔ Error during: … hf download black-forest-labs/FLUX.1-schnell │
╰──────────────────────────────────────────────────────────────────────────────╯
```

The full log it names ends with `Error: Access denied. Model 'black-forest-labs/FLUX.1-schnell' requires approval.`

Tip

`tt model pull <model>` downloads a model's weights ahead of time, so the serve itself is faster. It is also a quick way to confirm that your access to a gated model works.

Step 2: Serve with the CLI

Preview, then serve

`tt serve <model> --dry-run` shows how the model would be served without starting anything: the serving path, the device configuration, the container image, where the weights go, and the port.

```
tt serve Wan2.2-T2V-A14B-Diffusers --dry-run
```

```
✓ Detecting device configuration  p300x2 (override with --device)
                            tt serve Wan2.2-T2V-A14B-Diffusers — dry run
┌──────────────────┬──────────────────────────────────────────────────────────────────────────────────────────────────────┐
│ backend          │ tt-inference-server                                                                                  │
│ device           │ p300x2                                                                                               │
│ engines          │ media                                                                                                │
│ status           │ COMPLETE                                                                                             │
│ docker image     │ ghcr.io/tenstorrent/tt-media-inference-server:0.17.0-8c48a10                                         │
│ weights cache    │ /home/ttuser/.cache/huggingface                                                                      │
│ HF token         │ from the Hugging Face login store                                                                    │
│ port             │ 20000  (server default)                                                                              │
└──────────────────┴──────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

`tt serve <model>` serves the model. On first use it fetches the weights and pulls the container image, then opens the devices, loads the model, and starts the server, showing each stage as a live checklist. It returns once the endpoint answers requests:

```
tt serve Wan2.2-T2V-A14B-Diffusers
```

```
  ✓ host ready  tt-inference-server 0.25.0
  ✓ weights ready  Wan-AI/Wan2.2-T2V-A14B-Diffusers  16m 58s
  ✓ image ready  tt-media-inference-server:0.17.0  2m 13s
  ✓ container started  tt-inference-server-2f018ce0  3.7s
  ✓ server started  4.5s
  ✓ Tenstorrent device opened  4 chips  4.3s
  ✓ model loaded  2/2  5m 52s
  ✓ endpoint answering
╭─ Wan2.2-T2V-A14B-Diffusers ready ──────────────────────────────────────────╮
│  endpoint  http://127.0.0.1:20000/v1                                       │
│  Logs · tt model logs Wan2.2-T2V-A14B-Diffusers --follow                   │
│  Stop · tt model stop Wan2.2-T2V-A14B-Diffusers                            │
╰────────────────────────────────────────────────────────────────────────────╯
```

On a Wormhole TT-LoudBox or TT-QuietBox the device line reads `t3k` and `Tenstorrent device opened` reports 8 chips, one per n300 half; the full first serve of this model took about 32 minutes there. `tt serve --dry-run` is the quickest way to see which configuration `tt` detected.

Send a request

The endpoint speaks the OpenAI API, so any OpenAI-compatible client can use it. Point the client at `http://localhost:20000/v1` with any value for the API key. Which route you call depends on the kind of model:

Kind of model	Route	What you send
Large language and vision-language models	`/v1/chat/completions`	a list of `messages`
Speech recognition	`/v1/audio/transcriptions`	the audio file, base64-encod
Text to speech	`/v1/audio/speech`	the `text` to speak
Image generation	`/v1/images/generations`	a `prompt`, with optional `num_` and `height`
Video generation	`/v1/videos/generations`	a `prompt`; the reply is a job t
Embeddings	`/v1/embeddings`	the `input` text

For an LLM, `tt model curl` sends a chat completion without any JSON on your part and prints the server's JSON reply, and `--print` shows the equivalent `curl` command for your own scripts:

```
tt model curl "In one sentence, what is Tenstorrent?" --max-tokens 2048
```

For the other kinds of model, call the route directly; `tt model curl` always posts to `/v1/chat/completions`, so against an image, video, audio, or embedding server it returns `{"detail":"Not Found"}`. Video generation runs as a job: the request returns at once with the job's `id` and `status`, you poll the job until it is `completed`, and then you download the result.

```
# Submit the job; the reply includes its id and status
curl -s http://localhost:20000/v1/videos/generations \
  -H "Content-Type: application/json" \
  -d '{"prompt": "A red fox trotting through fresh snow in a pine forest, soft morning light,
camera tracking alongside"}'

# Poll until status is "completed", then download the clip
curl -s http://localhost:20000/v1/videos/generations/<job_id>
curl http://localhost:20000/v1/videos/generations/<job_id>/download --output fox.mp4
```

Manage what is running

```
tt model ps              # what is being served, on which port, and whether it is healthy
tt model logs <model>    # the server's log (--follow to stream it)
tt model stop <model>    # stop the server; the weights stay cached
```

`tt serve` takes `--port` to serve on a port other than 20000, and `--device` to override the detected configuration. Anything it does not recognize after `--` is passed through to the server itself.

Step 3: Serve with TT-Studio

TT-Studio serves the same catalog through a web application. It deploys the model you choose and gives you a browser interface for it: chat with knowledge bases for language models, image and video generation for diffusion models, speech-to-text, object detection, and a voice agent. Every model it deploys is also served on an API endpoint, so applications can use it at the same time.

Start TT-Studio and deploy a model

`tt serve <model> --studio` deploys a model with TT-Studio instead of directly. TT-Studio takes port 3000 and the devices for itself, so stop anything `tt serve` or `tt launch` started and reset the devices first:

```
tt model stop Wan2.2-T2V-A14B-Diffusers
tt device reset
tt serve Wan2.2-T2V-A14B-Diffusers --studio
```

On first use `tt` fetches the latest TT-Studio release, and TT-Studio asks you to accept its terms and offers a `tt-studio` shell shortcut. It then prints its web UI address and, once the model is deployed, the model's endpoint:

```
┌─ TT Studio is ready ──────────────────────────────────────────────────────┐
│  URL            ● http://localhost:3000                                   │
│  Hardware       QuietBox (QB2) · 4 device(s)                              │
└───────────────────────────────────────────────────────────────────────────┘

┌─ Wan2.2-T2V-A14B-Diffusers is ready ──────────────────────────────────────┐
│  Chips      0, 1, 2, 3                                                    │
│  Endpoint   ● http://localhost:20000/v1/videos/generations                │
│  Web UI     http://localhost:3000/models-deployed                         │
└───────────────────────────────────────────────────────────────────────────┘
```

Without a model name, `tt serve --studio` lists everything TT-Studio can deploy on your system and asks which one you want.

Deploy from the browser instead

Open `http://localhost:3000`, go to the models page, and pick a model from the list, which is filtered to what your hardware supports. The **Models Deployed** page lists every running model with its health, the chips it occupies, and its log.

Use the model

The Models Deployed page links to the interface for the model's kind. The endpoint in the ready box serves API clients at the same time, and `tt model curl` and `tt launch` work against it exactly as they do against a model started by `tt serve`.

Stop

`tt model stop <model>` stops a model that TT-Studio deployed and takes TT-Studio down with it, and takes it down even when no model is deployed; settings, weights, chat history, and knowledge bases are kept, but TT-Studio's own run log is not, so read it before you stop if a deploy went wrong. The `tt-studio` shortcut manages TT-Studio directly:

```
tt-studio --stop-model <model>   # stop one model and keep TT-Studio running
tt-studio --stop                 # stop TT-Studio, keeping its data
```

Serve a Community Model

Community bundles are models that anyone has packaged for Tenstorrent hardware and published on the Hugging Face Hub. `tt model list` includes the ones Tenstorrent has reviewed; `--include-unverified` shows the rest. They are served through the same commands, with the bundle's repository id in place of a catalog name:

```
tt model list --community                # bundles that fit your hardware
tt model info <namespace>/<name>         # the bundle's manifest and a compatibility verdict
tt serve <namespace>/<name>              # served through tt-model-manager
```

`tt` checks that the bundle was built for your chip architecture and that your system has enough chips before it serves, and asks for confirmation before it serves a bundle that has not been verified. Tenstorrent does not test or maintain community bundles. If you have brought a model up on Tenstorrent hardware yourself, the tt-model-manager documentation explains how to package and publish it so that anyone can serve it the same way.

Troubleshooting

The model I want is not in `tt model list`

The list is filtered to your detected hardware. `tt model list --all` shows every model on every device, and `tt model info <model>` says why one is left out for yours. A model that is validated on a smaller configuration than you have, such as a single-chip model on a TT-QuietBox 2, is listed and runs on part of the system.

The serve stops at `downloading weights` with "requires approval"

The model is gated and your Hugging Face account has not been granted access to it yet. Accept the license on the model's Hub page, wait for the owner to approve the request if the model requires that, and serve again. Make sure `tt model login` is for the same account.

`tt serve` fails with "Access to model ... is restricted"

The model is gated and your account has not accepted its license yet, or the login on this machine is for a different account. Accept the license on the model's Hub page, check `tt model login`, and serve again. `tt model logs <model>` shows the server's own output for any failed start.

A serve or a TT-Studio deploy fails with "Try resetting the board"

A model server that was stopped or crashed can leave the devices in a state the next server cannot use. Reset every device, confirming with `y` when asked, and serve again:

```
tt device reset
```

`tt serve` says the model did not report ready within 60 minutes

`tt serve` waits up to an hour for the endpoint to answer, then stops watching and leaves the server running. A cold start of a large diffusion model, or of a 70B-class or larger LLM on a Wormhole system, compiles for your hardware and can be slow, so first check whether it is still making progress: `tt model logs <model> --follow` shows the server's output, and `tt model ps` reports it

`healthy` once it answers. If the log shows an error instead, stop the server with `tt model stop <model>`. To wait longer next time, set `TT_SERVE_READY_TIMEOUT` to a number of seconds before you serve; on Wormhole, `TT_SERVE_READY_TIMEOUT=5400` is a safe default for the largest models.

A TT-Studio deploy stays at 0%

If the **Models Deployed** page shows "Starting deployment... 0%" for more than a few minutes, `tt model ps` lists nothing, and `docker ps` shows only the six `tt_studio_*` containers, the deploy job failed before it started a container and TT-Studio did not report it. The reason is in `~/.local/share/tenstorrent/tools/tt-studio-v*/src/logs/model_run.log`; `invalid choice` for the model name means TT-Studio's bundled inference server does not support that model yet, so pick another. Read the log first: `tt model stop <model>` removes it when it stops TT-Studio.

TT-Studio does not start because port 3000 is in use

TT-Studio needs port 3000 and does not move to another one. Stop whatever is using it first, for example a web UI started with `tt launch`: `tt launch stop openwebui`.

I am running out of disk space

`tt model rm <model>` removes a model's local artifacts and keeps its weights; add `--include-weights` to remove those too. The weights live under `~/.cache/huggingface`.

What to Do Next

Now that your model is running:

- **Connect an application to it** with `tt launch`. Open WebUI and AnythingLLM give you a chat interface in the browser; OpenCode, Aider, pi, Qwen Code, and Hermes Agent turn a tool-calling LLM into a coding agent in your terminal. `tt launch list` shows which are available on your machine, and the client launchers guide covers what each one needs and how to connect to a server on another machine.
- **Watch it in real time** — run `tt-toplike` in a second terminal window. TT-Toplike shows live core activity, power draw, and memory traffic on the chips while they serve requests.
- **Use it as a coding assistant** — the Coding Assistant with Aider lesson walks through pairing a local endpoint with Aider for on-device pair programming.

- **Build agents on top of it** — if you have a TT-QuietBox® 2, the Local AI Agents on QuietBox 2 lesson covers tool-calling pipelines and multi-agent workflows against your local endpoint.
- **Generate images and video at scale** — if you have a TT-QuietBox® 2, TT-Local-Generator runs alongside inference with its own queue and gallery.
- **Check supported models** — the tt-inference-server repository is the authoritative source for which models are validated on each hardware generation, and the catalog `tt model list` shows is built from it.

Need Additional Support?

If you encounter any issues, or have a question that isn't covered in the documentation, please raise a support request. Our team will review your request and provide assistance.

Tenstorrent Documentation

No hardware yet? Get access to Tenstorrent silicon right in your browser with Tenstorrent Cloud Console

GET STARTED

Installing Tenstorrent Software

Set up drivers, firmware, and the TT-Metalium environment on your Tenstorrent hardware, then verify your install.

TT-VSCODE-TOOLKIT

Learn by Building

Get hands-on with a guided learning path — from making your first inference to building full applications with LLMs.

Popular Destinations

TT-Studio

Point-and-click web UI for deploying and interacting with models.

TT-Inference-Server

Deploy and serve models — the canonical source for validated model support.

TT-Metalium

Low-level C++ SDK for writing custom kernels on Tensix cores.

Get Started →

Overview

How TT-Forge, TT-NN, and TT-Metalium fit together — and which to use when.

Installing Tenstorrent Software

Install drivers, firmware, and the TT-Metalium environment, then verify your setup.

Manual Installation

Manually install system dependencies, drivers, and the software stack step by step.

Serve Models

Serve LLMs, diffusion, speech, and vision models with the tt CLI or TT-Studio, and get an OpenAI-compatible endpoint.

Learn by Building

Get hands-on with guided, end-to-end lessons in the TT-VSCode-Toolkit.

Software →

TT-Forge™

End-to-end MLIR compiler. Compile models from PyTorch, JAX, and ONNX.

TT-XLA

PJRT-based frontend for running JAX and OpenXLA models on Tenstorrent hardware.

TT-NN™

High-level Python API of pre-optimized neural-network operations.

TT-Lang™

Python DSL for authoring fused custom operations.

TT-MLIR™

MLIR-based backend compiler infrastructure that lowers models onto Tenstorrent hardware.

TT-Metalium™

Low-level C++ SDK for writing custom kernels on Tensix cores.

Cloud-Native Support

Run Tenstorrent accelerators on Kubernetes — install, configure, and operate the device stack.

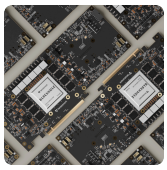
Hardware →

BLACKHOLE



TT-QuietBox® 2 (Blackhole)

Next-generation liquid-cooled Blackhole workstation.



Blackhole® PCIe Cards

Infinitely scalable, high-performance AI processing.



TT-QuietBox® (Blackhole)

Dead-quiet, liquid-cooled desktop AI.



Tenstorrent Galaxy® (Blackhole)

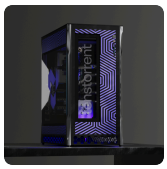
Scalable, enterprise-class AI server.

WORMHOLE



Wormhole™ PCIe Cards

Flexible, scalable processors built with Tensix Cores™.



TT-QuietBox® (Wormhole)

Dead-quiet desktop AI.



TT-LoudBox™ (Wormhole)

High-throughput workstation AI.



Tenstorrent Galaxy® (Wormhole)

Scalable, enterprise-class AI server.

Learn End-to-End through TT-VSCode-Toolkit →

Your First Inference

Set up the toolkit, detect hardware, and run your first model end-to-end.

Serving & APIs

Production inference with tt-inference-server and vLLM, plus image and video generation.

Applications

Real-world apps: coding assistants, AI agents, and video generation on QuietBox 2.

Custom Training

Datasets, fine-tuning, multi-device training, and training from scratch.

Cookbook

Hands-on projects: Game of Life, audio processing, fractals, and particle sims.

Compilers & Frameworks

Image classification with TT-Forge, JAX and PyTorch/XLA, and an intro to TT-Lang.

CS Fundamentals

RISC-V architecture, memory hierarchy, parallelism, synchronization, and complexity.

Advanced

Model bring-up bounties and exploring TT-Metalium at the kernel level.

Deployment

Deploy the toolkit and your work to the cloud with Koyeb.

Tools →

TT-Inference-Server

Deploy and serve models — the canonical source for validated model support.

TT-Studio

Point-and-click web UI for deploying a model — the fastest path for new users.

TT-SMI

Command-line device telemetry and firmware info — ships with the software stack.

TT-Toplike

Real-time terminal monitor: power, temperature, current, and core activity.

TT-NN Visualizer

Inspect model execution: graphs, memory plots, tensor and buffer views.

TT-Topology

Configure Ethernet routing on multi-card QuietBox and LoudBox systems.

TT-VSCode-Toolkit

Guided VS Code learning environment — browse tracks in the Learn section above.

TT-Blacksmith

Optimized ML training recipes for Tenstorrent hardware.

Explore the Ecosystem →

TT-Awesome

A curated directory of 115+ community and official projects, models, kernels, and research across 12 categories.

🪐 Planet Tenstorrent

The latest across the ecosystem — releases, papers, talks, videos, and demos, aggregated in one feed.

Tenstorrent on GitHub

All public source repositories — the software stack, compilers, tools, and demos.

Resources →



Request Support

Submit a support ticket and our team will help.



FAQ

Learn more about our hardware and software.



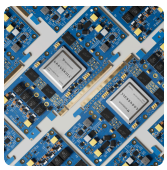
Contact Us

Get in touch with the Tenstorrent team.

Join Our Discord

Join the Tenstorrent developer community.

Legacy



Grayskull™

Tenstorrent's first-generation Tensix processor.



T7000

Earlier-generation Tenstorrent system.



T1000

Earlier-generation Tenstorrent system.

Installing Tenstorrent Software

This guide takes you from a fresh Tenstorrent system to one that is ready to run models. You will install the Tenstorrent CLI, `tt`, use it to update your system to the latest stable Tenstorrent software, and confirm your hardware is visible.

`tt` is the primary interface for operating your Tenstorrent hardware. If you want to develop against Tenstorrent's open-source repositories instead, start with TT-Metalium™, TT-NN™, or TT-Forge™.

Before You Begin

Note

Setting up your new TT-QuietBox® 2? Follow the Setup Guide.

Before you begin, ensure your host machine meets the following prerequisites:

- The host machine has an internet connection. `tt` downloads packages from PyPI and GitHub, the Tenstorrent package repositories, and model weights from the Hugging Face Hub.
- The host machine has a supported operating system installed (see below).
- You have administrator (`sudo`) privileges on the host machine. The driver, firmware tooling, and HugePages configuration are installed as root.
- The system is physically unboxed and set up according to your product's installation manual and safety guidelines.

Supported Operating Systems

Tenstorrent recommends Ubuntu LTS for all Tenstorrent software.

Operating system	Status
Ubuntu 22.04 LTS, Ubuntu 24.04 LTS	Supported
Other Linux distributions	Untested and considered experimental
macOS, Windows, WSL	Not supported. There is no driver, so nothing hardware-related work

BIOS Requirements

The BIOS for your host motherboard should configure the PCIe AER Reporting Mechanism to be set to `OS First`. Tenstorrent's TT-SMI software requires this setting to function properly.

Important

If you are using a TT-QuietBox, you do not have to worry about setting this option. It is set to `OS First` by default.

If you update or reset your BIOS for any reason, you must reconfigure the PCIe AER Reporting Mechanism setting to `OS First` to ensure TT-SMI functions correctly. This setting is typically located in the **Motherboard Information** section of your BIOS.

Install the Tenstorrent CLI Dependencies

The Tenstorrent CLI, `tt`, is a Python application. It needs Python 3.10 or newer with the `venv` module, plus `git` and `curl`. Install them with your system package manager. On Ubuntu:

```
sudo apt update && sudo apt install -y python3 python3-venv python3-pip git curl ca-certificates
```

`tt` must live in its own isolated Python environment so that it can safely update itself later. Tenstorrent recommends uv to manage that environment. Follow the setup instructions at https://docs.astral.sh/uv/ to install `uv`, then open a new terminal or source your shell configuration file (`source ~/.bashrc` for `bash`) so that `uv` is on your `PATH`.

Note

If you already use pipx, it works too. Install `tt` with `pipx install tenstorrent` in the next step instead of the `uv` command. Other installation options are described in the tt-cli developer notes.

Step 1: Install the Tenstorrent CLI

Note

If you're starting with a fresh TT-QuietBox® 2, we automatically install the Tenstorrent CLI for you. To check, run `tt --version` in your terminal. If you get a response, move to Step 2 below.

Install `tt` as a `uv` tool and make sure it is on your `PATH`:

```
uv tool install tenstorrent
uv tool update-shell
```

Open a new terminal or source your shell configuration file again, then confirm `tt` runs:

```
tt --version
```

The first time you run `tt` in a terminal it asks once whether you want to share anonymous usage telemetry. Please consider saying yes. Telemetry tells Tenstorrent which commands people use, how long they take, and which ones fail, and that is what guides what gets fixed and built next. It is strictly opt-in and records no prompts, file paths, hostnames, usernames, or IP addresses. You can see exactly what would be sent before you decide: set `TT_TELEMETRY_LOG_FILE=/tmp/tt-events.jsonl` on any command and read the file, which works without opting in and uploads nothing. You can change your answer at any time with `tt config set telemetry.enabled true` or `false`. The telemetry policy lists everything that is collected.

Run `tt` to see the command groups. The ones used in this guide are `update` and `device`; `model`, `serve`, and `launch` take over once the stack is installed.

Step 2: Install the Software Stack with `tt update`

Tenstorrent's system software stack consists of several components that are regularly tested together to ensure stability: device firmware, the kernel-mode driver (TT-KMD), HugePages configuration, TT-SMI, TT-Flash, and a container runtime for serving models. `tt update` brings your system up to the latest stable, tested combination of those components in one command. On a fresh machine it installs everything. On a machine that already has Tenstorrent software it changes only what is out of date, so you can rerun it at any time.

```
tt update
```

`tt update` installs the Tenstorrent tools into isolated environments under `~/.local/share/tenstorrent/`, where they cannot conflict with your own Python environment, then runs the matching tt-installer release non-interactively to install the driver, firmware, HugePages configuration, and Docker if no container runtime is present.

Required

Using sudo is required, so you must enter your user's password when prompted.

`tt update` also asks you to confirm before it flashes firmware, because flashing resets the devices. Stop any running model workloads first, then answer **y**.

On a TT-QuietBox 2 with a fast connection the whole update takes a few minutes.

Bring `tt` itself up to date

The installer that `tt update` runs also installs the `tt` release that was tested with that set of components, which can be older than the one you installed in Step 1. This page documents the latest release, so before you move on, put `tt` back on it:

```
tt self update
```

Keep both halves current over time: `tt update` for the system software and tools, and `tt self update` for `tt` itself. `tt` checks for a newer release once a day and prints a notice when one is available.

Reboot after the first install

`tt update` never reboots your machine for you. If this is the first time the driver was installed on this machine, or HugePages configuration changed, **reboot now** before continuing:

```
sudo reboot
```

If `tt update` installed Docker, it also adds your user to the `docker` group. That takes effect when you log in again, which the reboot takes care of.

Step 3: Verify Your Hardware

After rebooting, confirm the driver is loaded and that every Tenstorrent device in your system is visible:

```
tt device status
```

You should see one row per chip, with the board type, PCIe bus address, and live telemetry. The **Board** column tells you which generation you have: `p100`, `p150`, and `p300` boards are Blackhole; `n150` and `n300` boards are Wormhole. A few later steps apply to one generation only and say so. Here is an example from a TT-QuietBox 2 with four Blackhole p300 cards:

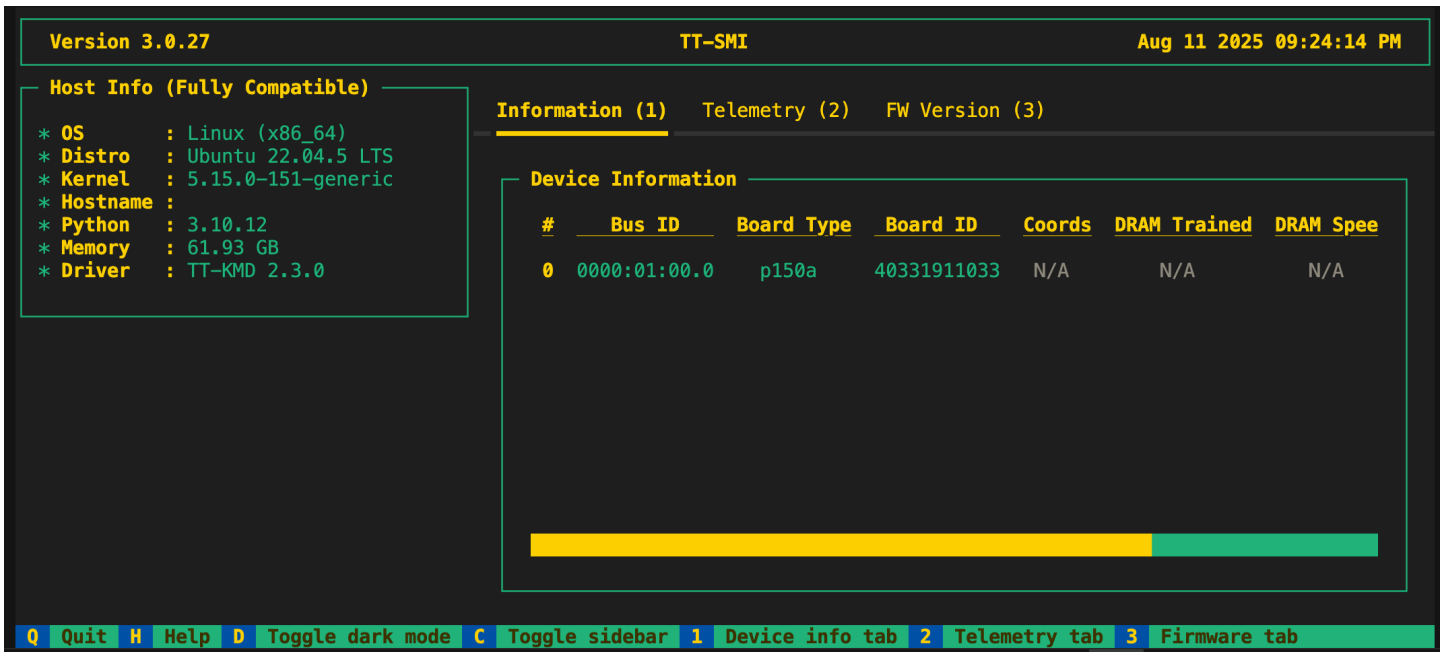
```
                Tenstorrent devices (driver: TT-KMD 2.11.0)
┏━━━┳━━━━━━━┳━━━━━━━━━━━━━┳━━━━━━━━┳━━━━━━━━┳━━━━━━━━━┳━━━━━━━━━┳━━━━━━┓
┃ # ┃ Board ┃ Bus ID      ┃ Temp   ┃ Power  ┃ AICLK   ┃ Voltage ┃ DRAM ┃
┡━━━╇━━━━━━━╇━━━━━━━━━━━━━╇━━━━━━━━╇━━━━━━━━╇━━━━━━━━━╇━━━━━━━━━╇━━━━━━┩
│ 0 │ p300c │ 0000:01:00.0 │ 43.9 °C │ 31.0 W │ 800.0 MHz │ 0.71 V  │ True │
│ 1 │ p300c │ 0000:02:00.0 │ 44.4 °C │ 17.0 W │ 800.0 MHz │ 0.71 V  │ True │
│ 2 │ p300c │ 0000:03:00.0 │ 43.6 °C │ 16.0 W │ 800.0 MHz │ 0.72 V  │ True │
│ 3 │ p300c │ 0000:04:00.0 │ 45.1 °C │ 16.0 W │ 800.0 MHz │ 0.72 V  │ True │
└───┴───────┴──────────────┴────────┴────────┴─────────┴─────────┴──────┘
```

A Wormhole n300 card carries two chips and shows two rows, `n300 L` with the card's PCIe bus address and `n300 R` with `N/A`, because the second chip is reached through the first. A TT-LoudBox or Wormhole TT-QuietBox with four n300 cards therefore shows eight rows:

```
                Tenstorrent devices (driver: TT-KMD 2.11.0)
┏━━━┳━━━━━━━┳━━━━━━━━━━━━━┳━━━━━━━━┳━━━━━━━━┳━━━━━━━━━━┳━━━━━━━━━┳━━━━━━━┓
┃ # ┃ Board ┃ Bus ID      ┃ Temp   ┃ Power  ┃ AICLK    ┃ Voltage ┃ DRAM  ┃
┡━━━╇━━━━━━━╇━━━━━━━━━━━━━╇━━━━━━━━╇━━━━━━━━╇━━━━━━━━━━╇━━━━━━━━━╇━━━━━━━┩
│ 0 │ n300 L │ 0000:31:00.0 │ 45.9 °C │ 28.0 W │ 1000.0 MHz │ 0.89 V  │ True  │
│ 1 │ n300 L │ 0000:4b:00.0 │ 44.8 °C │ 27.0 W │ 1000.0 MHz │ 0.88 V  │ True  │
│ 2 │ n300 L │ 0000:b1:00.0 │ 44.4 °C │ 29.0 W │ 1000.0 MHz │ 0.9 V   │ True  │
│ 3 │ n300 L │ 0000:ca:00.0 │ 43.1 °C │ 29.0 W │ 1000.0 MHz │ 0.9 V   │ True  │
│ 4 │ n300 R │ N/A          │ 37.5 °C │ 24.0 W │ 1000.0 MHz │ 0.87 V  │ True  │
│ 5 │ n300 R │ N/A          │ 36.4 °C │ 20.0 W │ 1000.0 MHz │ 0.91 V  │ True  │
│ 6 │ n300 R │ N/A          │ 37.4 °C │ 24.0 W │ 1000.0 MHz │ 0.91 V  │ True  │
│ 7 │ n300 R │ N/A          │ 36.1 °C │ 23.0 W │ 1000.0 MHz │ 0.9 V   │ True  │
└───┴────────┴──────────────┴─────────┴────────┴────────────┴─────────┴───────┘
```

The number of rows should match the number of Tenstorrent chips installed in your system: one per Blackhole card, two per n300 card. For more detail on a device, such as its firmware versions and board ID, run `tt device info`. For a live, interactive view, run `tt smi`, which hands your terminal to the TT-SMI interface. Press **Q** to leave it.

Here is an example of the TT-SMI interface for a system containing a single p150a device:



Important

If the number of listed devices does not match what you expect, see Troubleshooting below. If the problem persists, contact support and we will assist you.

Congratulations, your system software is installed and your hardware is ready to use.

Step 4 (Wormhole only): Configure the Mesh Topology

Skip this step on Blackhole systems and on single-card Wormhole systems.

A Wormhole TT-LoudBox or TT-QuietBox connects its n300 cards to each other over Ethernet, and the models in the catalog expect those links to form a mesh. `tt update` installs the driver and firmware but does not configure the mesh, and a system can arrive, or come back from a firmware update, with the chips **Isolated**. Check first:

```
tt-topology -ls
```

If it reports `Mesh`, continue to the next page. If it reports `Isolated`, or the command is not found, run the script below once. It installs TT-Topology into a temporary environment, resets every chip, and writes the mesh configuration. It needs no `sudo`, takes about two minutes, and ends with "Completed reset on 8 chips" on a four-card system.

```
TMP_DIR=$(mktemp -d); (trap 'echo "---"; echo "Cleaning up..."; if type deactivate &>/dev/null; then deactivate; fi; echo "Removing temporary directory: $TMP_DIR"; rm -rf "$TMP_DIR"; cd; echo "Cleanup complete."' EXIT; trap 'echo -e "\033[0;31m!!! ERROR: Failed to configure mesh topology\033[0m"' ERR; set -e; cd "$TMP_DIR"; echo "Working in temporary directory: $TMP_DIR"; echo "---"; echo "Creating Python virtual environment..."; python3 -m venv tt-topology-venv; source tt-topology-venv/bin/activate; echo "Virtual environment activated."; echo "---"; echo "Installing tt-topology from git..."; pip install --quiet git+https://github.com/tenstorrent/tt-topology.git; echo "tt-topology installed."; echo "---"; echo "Running tt-topology command. This may take a moment..."; tt-topology -l mesh; echo "---"; echo "Script finished successfully.";)
```

Run `tt-topology -ls` again and confirm `Mesh`. Serving a model without this step stops at system validation with `ModelSpec requires SystemTopology.MESH, detected ISOLATED`.

Troubleshooting

`tt device status` reports no devices

If `tt device status` exits with a message that the driver may not be installed or no card is seated:

1. If you have not rebooted since `tt update` installed the driver, reboot now.
2. Run `lspci -d 1e52:` to check whether the card has enumerated on the PCIe bus. If it returns nothing, the host cannot see the card at all. Each Blackhole card and each n150 card appears once; an n300 card also appears once, because its second chip has no PCIe address of its own.
3. For Blackhole PCIe products such as p100 and p150, verify the card is being supplied power by checking that the fan spins and the green power LED illuminates when the system boots. If the light is on, try resetting the card with `tt device reset`.
4. Confirm the BIOS setting described in BIOS Requirements.

Docker commands are refused with a permission error

`tt` talks to Docker as your user, never through `sudo`. If `tt update` just installed Docker, log out and back in (or reboot) so that your membership in the `docker` group takes effect. If Docker was already installed before you ran `tt update`, the installer leaves it as it found it and does not add you to the group. Add yourself, then log out and back in:

```
sudo usermod -aG docker $USER
```

You can check with `docker run hello-world`.

Getting help

`tt report bundle` collects a redacted support bundle with your environment details, a TT-SMI snapshot, and recent logs, and opens a prefilled email to Tenstorrent support with it attached. `tt report issue` opens a prefilled GitHub issue on the tt-cli repository.

Alternative Installation Methods

`tt update` is the recommended way to install the system software, and it runs tt-installer for you with the tested versions. If you prefer to drive tt-installer interactively yourself and choose each of its options, you can run it directly:

```
/bin/bash -c "$(curl -fsSL https://tenstorrent.ai/install.sh)"
```

For advanced users or developers who prefer to install and configure each component by hand, refer to the manual software installation guide.

What's Next

With the software stack installed, `tt` takes you the rest of the way to a running model. `tt model list` shows the models validated for the hardware it detected, and `tt model info <model>` reports the disk space, host memory, and validation status of each one. Catalog models download their weights from the Hugging Face Hub, so log in once with `tt model login`; gated models such as Llama and Gemma also require accepting the license on the model's Hub page first. `tt serve <model>` then installs TT-Inference-Server, downloads the weights, and starts an OpenAI-compatible endpoint on port 20000, and `tt model curl "<prompt>"` sends it a first request. The first serve of a model downloads and compiles a lot, so expect it to take a while; later serves are much faster.

An endpoint is the foundation; `tt launch` puts an application on top of it. `tt launch openwebui` starts a chat interface in your browser, and `tt launch list` shows the other web UIs and terminal coding agents available on your machine. If you prefer a point-and-click experience, `tt serve <model> --studio` starts the model inside TT-Studio, a web application that serves models and gives you a browser interface for them in one place. The Serve Models guide walks through all of this in detail. Run `tt` at any time to see the command groups, and `tt <command> --help` for the options of each.

Where to Go Next

You now have the full stack installed. From here, pick the part of the stack that matches what you want to build:

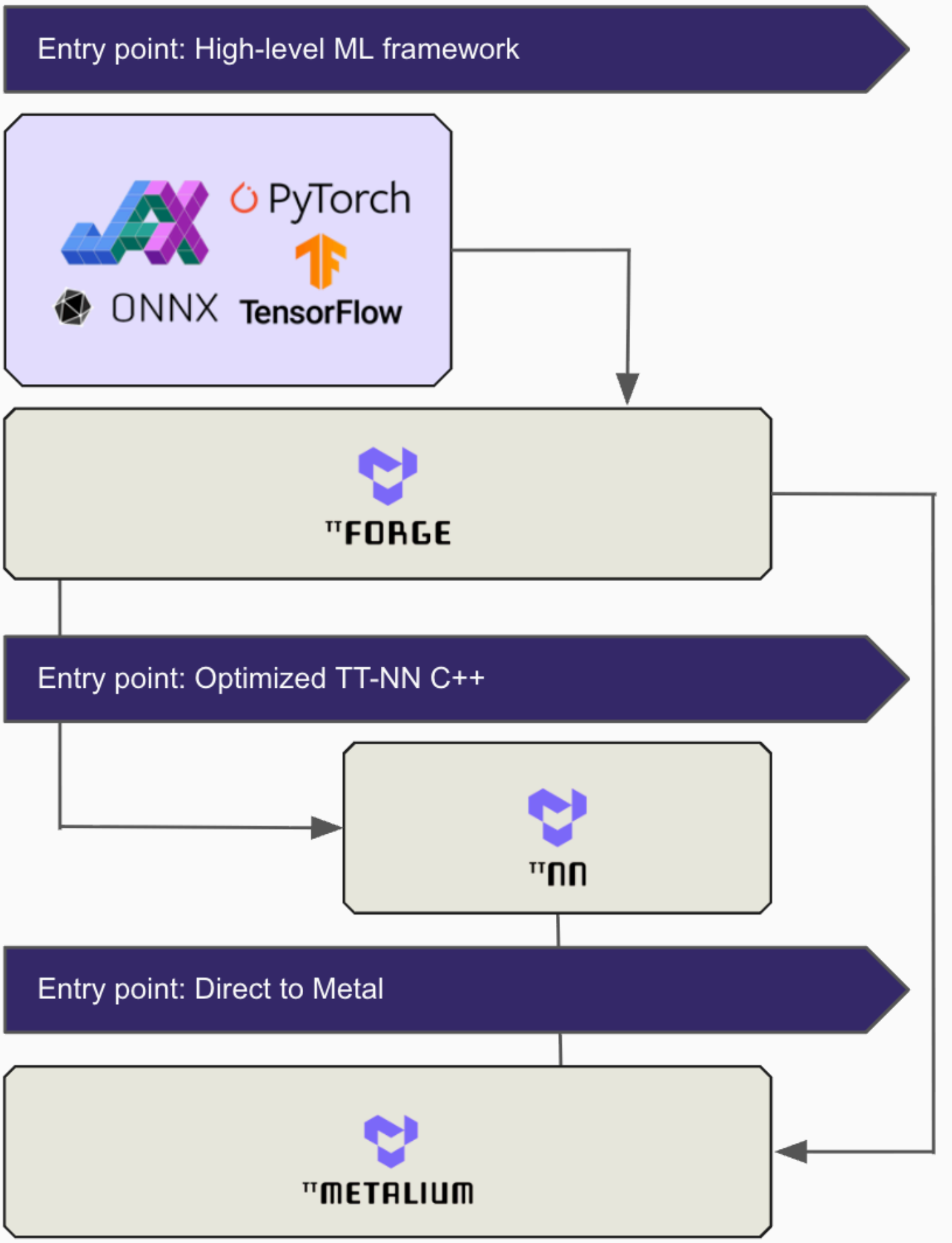
- **Serve more kinds of models** by following the Serve Models guide: choosing a model for your hardware, serving LLMs, diffusion, speech, and vision models from the CLI, and running the same models through TT-Studio's web interface.
- **Deploy LLMs in production** with TT-Inference-Server, the serving engine behind `tt serve`. Its repository is the authoritative source for which models are validated on your hardware.
- **Monitor your hardware** with `tt smi` for a status snapshot, or TT-Toplike for a live view of power, temperature, and core activity while a model serves requests.
- **Use a web interface** with TT-Studio, started through `tt serve <model> --studio`.
- **Understand the software stack**, including how TT-Forge™, TT-NN™, and TT-Metalium™ relate to each other, in the software stack overview.
- **Compile your own PyTorch, JAX, or ONNX models** with TT-Forge™.
- **Run and build models in Python** with TT-NN™, or **write custom kernels** with TT-Metalium™. `tt update` also installed a `tt-metalium` command that opens a shell inside the TT-Metalium container, with your home directory mounted and TT-NN ready to import; the image, about 4.5 GB, is pulled the first time you run it. Start with the Metalium guide to learn about the architecture.
- **Learn by building** with the hands-on lessons in the TT VS Code Toolkit.

Need Additional Support?

If you encounter any issues, or have a question that isn't covered in the documentation, please raise a support request. Our team will review your request and provide assistance.

Overview

This document provides developers and researchers with an overview of the main components of the Tenstorrent software stack: TT-Metalium™, TT-NN™, and TT-Forge™. You will learn about each tool's purpose, use cases, and where to find its source code.


Entry point: High-level ML framework
PyTorch
ONNX
TensorFlow
TTFORGE
Entry point: Optimized TT-NN C++
TTNN
Entry point: Direct to Metal
TTMETALIUM

TT-Forge™: The MLIR-Based Compiler

TT-Forge™ is Tenstorrent's Multi-Level Intermediate Representation (MLIR)-based compiler. It bridges high-level machine learning frameworks with the Tenstorrent software stack.

Interact with TT-Forge™ to compile models from frameworks such as PyTorch, JAX, and TensorFlow for execution on Tenstorrent hardware. It offers an automated, general path to run many types of model architectures without requiring custom kernel development. TT-Forge™ integrates with and lowers to TT-Metalium for hardware execution.

The main project is located in the tt-forge GitHub repository.

TT-NN™: A Python & C++ Neural Network OP library

TT-NN™ is a library of neural network operations that provides a user-friendly interface for running models on Tenstorrent hardware. It is designed to be intuitive for developers familiar with PyTorch.

Interact with TT-NN™ to run AI models using a familiar, high-level Python API without managing the complexities of the underlying hardware. TT-NN™ builds upon TT-Metalium™ and provides a stable set of pre-packaged, optimized operations. It is also available with a C++ API.

The TT-NN™ library is part of the tt-metal GitHub repository.

TT-Metalium™: Programming Tenstorrent Hardware

TT-Metalium™ is the low-level, open-source software development kit (SDK) that provides developers direct access to Tenstorrent hardware. It is a bare-metal programming environment designed for users who must write custom C++ kernels for machine learning or other high-performance computing workloads.

Interact with TT-Metalium™ when you require complete control over the hardware to optimize code for performance, explicitly manage memory, or implement novel operations not found in standard libraries. This environment exposes the RISC-V processors, the Network-on-Chip (NoC), and the matrix and vector engines within each Tensix core.

The main project is available in the tt-metal GitHub repository.

Running Models on Hardware

Once you have the software stack installed, these are the primary entry points for running models:

- **TT-Inference-Server** — manages Docker containers, model downloads, and serving configuration, and provides an OpenAI-compatible API endpoint. It is the authoritative source for which models are validated on each hardware generation.
- **TT-Forge Models** — the catalog of models that have been validated against TT-Forge™. The starting point when compiling a PyTorch, ONNX, or JAX model against the stack.
- **TT-Studio** — a web interface over TT-Inference-Server for users who prefer a point-and-click model deployment flow.

For monitoring your hardware while models run, TT-Toplike reads power, temperature, and core activity in real time from the chips.

Exploring our Developer Hub

To continue learning, visit and explore our Developer Hub. There you will find in-depth articles, and information on our software bounty program. You can also find tutorials and technical overviews on our YouTube channel.

Engaging with our Community

For real-time discussions, community Q&A, and to share your latest projects, we invite you to join our Discord channel. It's a great place to engage directly with other developers and members of the Tenstorrent community.

As always, if you have any questions, please raise a support request with our team. We're here to help, and are excited to see what you build.

PREVIOUS NEXT